End-to-End Exploratory Data Analysis (EDA) on the Titanic Dataset

Project Objective: To perform a comprehensive, step-by-step exploratory data analysis to understand the key factors that influenced survival on the Titanic. This notebook will serve as a complete guide, covering data loading, cleaning, analysis, feature engineering, and visualization, with theoretical explanations at each stage.

Code and Inference: The python codes and the inference of the codes are at the bottom of this page.

Theoretical Concept: What is Exploratory Data Analysis (EDA)?

Exploratory Data Analysis is the crucial process of performing initial investigations on data to discover patterns, spot anomalies, test hypotheses, and check assumptions with the help of summary statistics and graphical representations. It is not about formal modeling or hypothesis testing; rather, it is about getting to know your data before you start building models.

Why is it important?

1. **Understand the Data:** It helps you understand the variables and their relationships.
2. **Data Cleaning:** It reveals missing values, outliers, and other inconsistencies that need to be handled.
3. **Feature Selection:** It helps identify which variables are the most important for your problem (feature engineering and selection).
4. **Assumption Checking:** It allows you to check assumptions that are required for certain machine learning models (e.g., normality, linearity).

Libraries Used: Pandas and Seaborn

- **Pandas:** This is a powerful Python library for data manipulation and analysis. It provides data structures like DataFrames, which are essential for working with tabular data. We used Pandas to load the dataset, handle missing values, and perform various data transformations.
- **Seaborn:** Built on top of Matplotlib, Seaborn is a statistical data visualization library. It provides a high-level interface for drawing attractive and informative statistical graphics. We used Seaborn to create various plots like countplots,

histograms, boxplots, and barplots to visualize the distributions and relationships within the data.

Step 1: Setup - Importing Libraries

We'll start by importing the essential Python libraries for data manipulation (`pandas`, `numpy`) and visualization (`matplotlib`, `seaborn`).

Step 2: Data Loading and Initial Inspection

We'll load the dataset and take our first look at its structure, content, and overall health.

Step 3: Data Cleaning

Before analysis, we must handle the missing values we identified.

Theoretical Concept: Missing Value Imputation

Imputation is the process of replacing missing data with substituted values. The strategy depends on the data type and its distribution:

- **Numerical Data:** For skewed distributions (like `Age` and `Fare`), using the **median** is more robust than the mean because it is not affected by outliers.
- **Categorical Data:** A common strategy is to fill with the **mode** (the most frequent value).
- **High Cardinality/Too Many Missing Values:** For columns like `Cabin`, where most data is missing, imputing might not be effective. We could either drop the column or engineer a new feature from it (e.g., `Has_Cabin`).

Step 4: Univariate Analysis

We analyze each variable individually to understand its distribution.

Theoretical Concept: Univariate Analysis

This is the simplest form of data analysis, where the data being analyzed contains only one variable. The main purpose is to describe the data and find patterns within it.

- **For Categorical Variables:** We use frequency tables, bar charts (`countplot`), or pie charts to see the count or proportion of each category.
- **For Numerical Variables:** We use histograms (`histplot`) or kernel density plots (`kdeplot`) to understand the distribution, and box plots (`boxplot`) to identify the central tendency, spread, and outliers.

Step 5: Bivariate Analysis

Here, we explore the relationship between two variables. Our primary focus will be on how each feature relates to our target variable, `Survived`.

Theoretical Concept: Bivariate Analysis

This type of analysis involves two different variables, and its main purpose is to find relationships between them.

- **Categorical vs. Numerical:** To compare a numerical variable across different categories, we often use bar plots (`barplot`) that show the mean (or another estimator) of the numerical variable for each category. We can also use box plots or violin plots.
- **Categorical vs. Categorical:** We can use stacked bar charts or contingency tables (crosstabs).
- **Numerical vs. Numerical:** A scatter plot is the standard choice, with a correlation matrix being used to quantify the relationship.

Step 6: Feature Engineering

Now, we'll create new features from the existing ones to potentially uncover deeper insights and provide more useful information for a machine learning model.

Theoretical Concept: Feature Engineering

Feature engineering is the process of using domain knowledge to extract features (characteristics, properties, attributes) from raw data. A good feature should be relevant to the problem and easy for a model to understand.

Common Techniques:

1. **Combining Features:** Creating a new feature by combining others (e.g., `SibSp` + `Parch` = `FamilySize`).
2. **Extracting from Text:** Pulling out specific information from a text feature (e.g., extracting titles from the `Name` column).
3. **Binning:** Converting a continuous numerical feature into a categorical one (e.g., binning `Age` into groups like 'Child', 'Adult', 'Senior').

Step 7: Multivariate Analysis

Now we explore interactions between multiple variables simultaneously, including our new engineered features.

Step 8: Correlation Analysis

Step 9: y-profiling

YData-Profiling, formerly known as Pandas Profiling, is a Python package designed for generating detailed reports on datasets. It provides a comprehensive overview of the data, including statistics, distribution of values, missing values, and memory usage, making it a valuable tool for exploratory data analysis (EDA). The package supports various data types, including tabular, time-series, text, and image data, and can handle large datasets efficiently. It also offers features such as correlations, interactions, and visualizations to facilitate data understanding and analysis

Reference urls:

1. https://docs.profiling.ydata.ai/latest/
2. https://www.geeksforgeeks.org/data-analysis/unlocking-insights-with-exploratory-data-analysis-eda-the-role-of-ydata-profiling/
3. https://www.youtube.com/watch?v=5Otpw0jPw4s

Step 1: Setup - Importing Libraries

We'll start by importing the essential Python libraries for data manipulation (`pandas`, `numpy`) and visualization (`matplotlib`, `seaborn`).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# Using plot style for better aesthetics
sns.set_style('whitegrid')
```

```
!git clone 'https://github.com/kirangv/G4G_Source_data'
```

```
Cloning into 'G4G_Source_data'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), 22.32 KiB | 3.72 MiB/s, done.
```

```
df = pd.read_csv('/content/G4G_Source_data/Titanic-Dataset.csv')
```

```
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs	female	38.0	1	0	PC 17599

Next steps: Generate code with df | New interactive sheet

```
df.tail()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053

```
df.shape
```

```
(891, 12)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
```

```
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Interpretation of .info():

- The dataset contains 891 entries (rows, passengers) and 12 columns.
- Missing Values Identified: Age, Cabin, and Embarked have missing values. Cabin is missing a significant amount of data (~77%), which will require special attention.

```
df.describe()
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	89
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	3
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	4
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	1
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	3
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	51

Interpretation of .describe():

- Survived: About 38.4% of passengers in this dataset survived.
- Age: The age ranges from 4.2 months to 80 years, with an average age of about 30.
- Fare: The fare is highly skewed, with a mean of 32 but a median of only 14.45. The maximum fare is over $512, indicating the presence of extreme outliers.

```
df['Embarked'].value_counts()
```

Embarked	count
S	644
C	168
Q	77

dtype: int64

```
df['Cabin'].value_counts()
```

Cabin	count
G6	4
C23 C25 C27	4
B96 B98	4
F2	3
D	3
...	...
E17	1
A24	1
C50	1
B42	1
C148	1

147 rows × 1 columns

dtype: int64

```
# Identifying NULL values
df.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

We have NULL values present in variables - Age, Cabin and Embarked. Let's handle them 1 by 1.

```
# 1. Handle missing 'Age' Values:
# Since Age is considered a number (continious), we will impute the missing val
median = df['Age'].median()
df['Age'] = df['Age'].fillna(median)
```

```
# 2. Handle missing 'Embarked' Values:
# Since Embarked column has Categorical values, we will impute the missing valu
df['Embarked'].mode()[0]
```

```
'S'
```

```
mode = df['Embarked'].mode()[0]
df['Embarked'] = df['Embarked'].fillna(mode)
```

```
df.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	0
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	0

dtype: int64

```
# 3. Handle missing 'Cabin' Values:
# Since Cabin column has Categorical values, we will impute the missing values
df['Has_Cabin'] = df['Cabin'].notna().astype(int)
```

```
# We can now drop the 'Cabin' variable from the dataset since the new 'Has_Cabi
df.drop('Cabin', axis=1, inplace=True)
```

```
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs	female	38.0	1	0	PC 17599

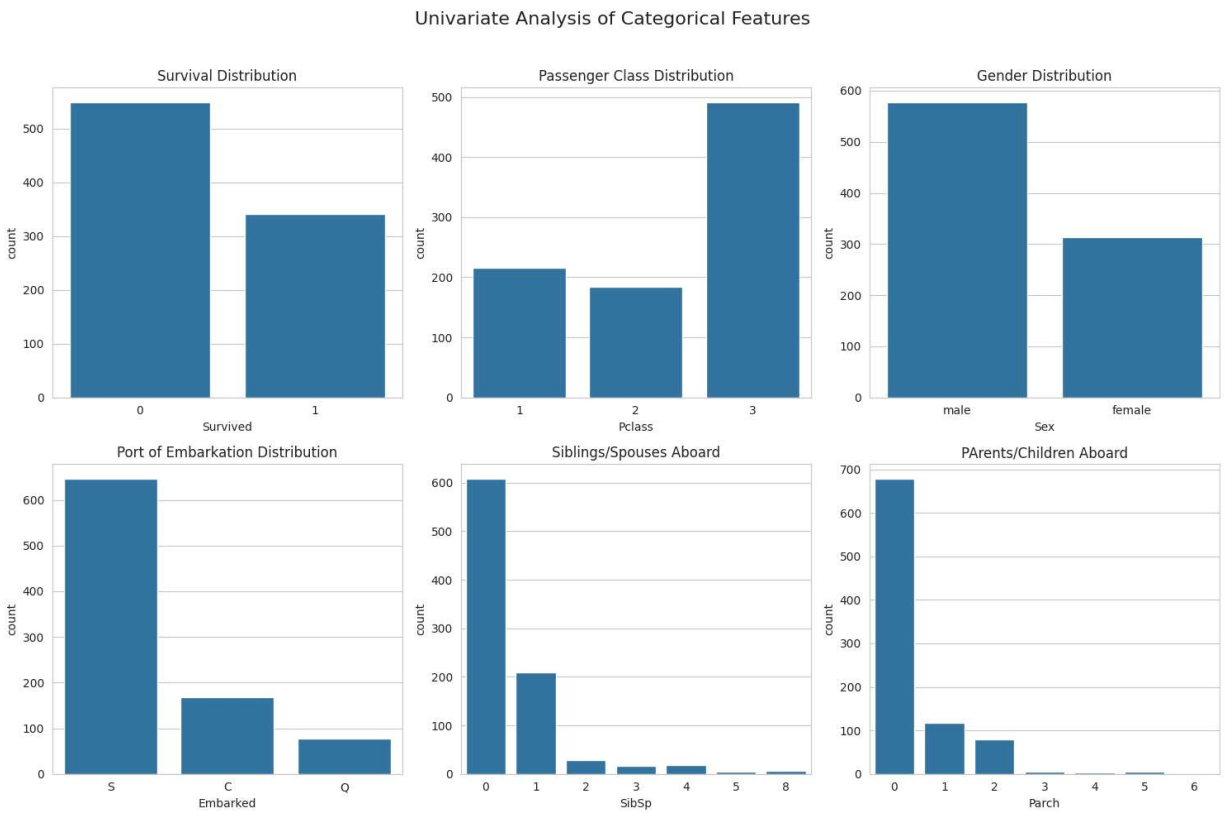
Next steps: Generate code with df | New interactive sheet

```
# Checking if there exists any Null / missing values
df.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	0
SibSp	0
Parch	0
Ticket	0
Fare	0
Embarked	0
Has_Cabin	0

dtype: int64

```
# Univariate Analysis
## Categorical Charts
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
fig.suptitle('Univariate Analysis of Categorical Features', fontsize=16)

sns.countplot(ax=axes[0,0], x='Survived', data=df).set_title('Survival Distribu
sns.countplot(ax=axes[0,1], x='Pclass', data=df).set_title('Passenger Class Dis
sns.countplot(ax=axes[0,2], x='Sex', data=df).set_title('Gender Distribution')
sns.countplot(ax=axes[1,0], x='Embarked', data=df).set_title('Port of Embarkati
sns.countplot(ax=axes[1,1], x='SibSp', data=df).set_title('Siblings/Spouses Abc
sns.countplot(ax=axes[1,2], x='Parch', data=df).set_title('PArents/Children Abc

plt.tight_layout(rect=[0,0,1,0.96])
plt.show()
```



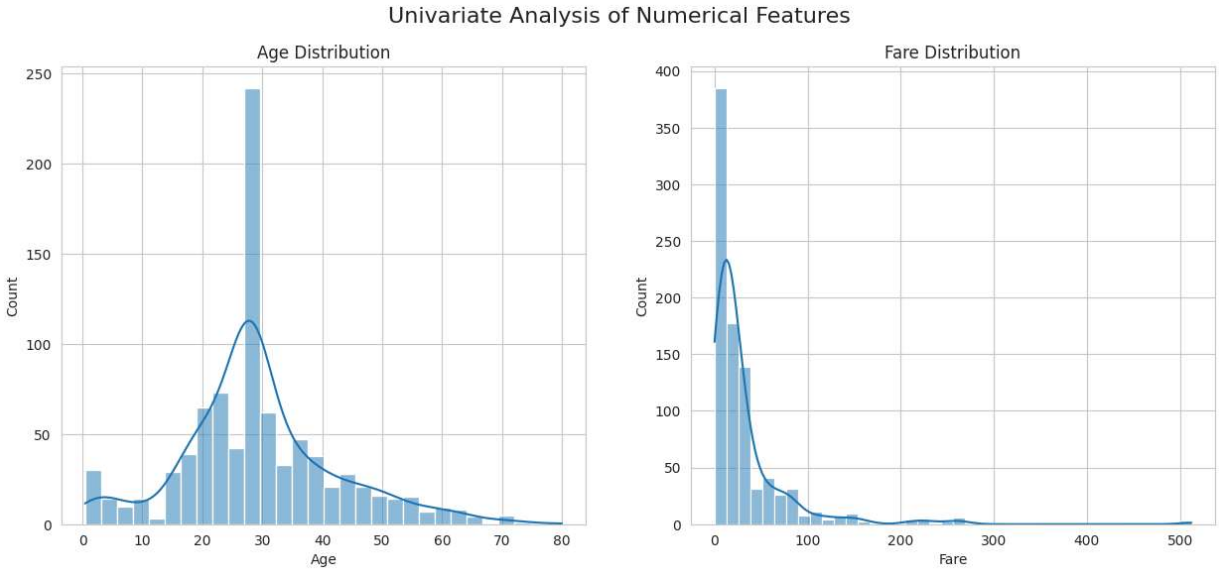
Inference of Univariate Analysis of Categorical Charts

- **Survival**: Most passengers (over 500 of 891) did not survive.
- **Pclass**: The 3rd class was the most populated, followed by 1st and then 2nd.
- **Gender**: There were significantly more males than females.
- **Embarked**: The vast majority of passengers embarked from Southampton ('S').

- **SibSp & Parch**: Most passengers traveled alone.

```
## Univariate Analysis for Numerical Columns
print("\nAnalyzing numerical features:")
fig, axes = plt.subplots(1, 2, figsize=(15,6))
fig.suptitle('Univariate Analysis of Numerical Features', fontsize=16)

# Plotting Age distribution
sns.histplot(ax=axes[0], data=df, x='Age', kde=True, bins=30).set_title('Age Di

# Plotting Fare distribution
sns.histplot(ax=axes[1], data=df, x='Fare', kde=True, bins=40).set_title('Fare

plt.show()
```

```
Analyzing numerical features:
```



Inference of Univariate Analysis of Numerical Charts

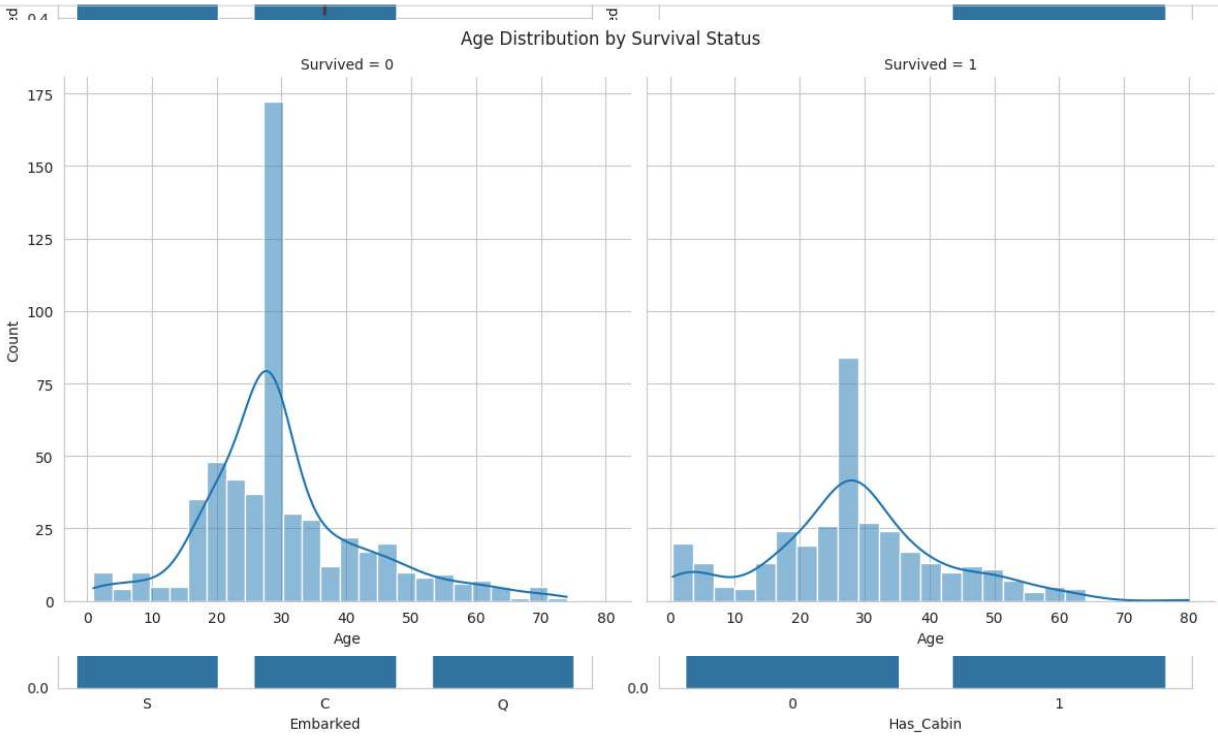
- **Age**: The distribution peaks around the 20-30 age range. Since we filled missing values with the median (28), this contributes to the height of that central bar.
- **Fare**: The distribution is heavily right-skewed, confirming that most tickets were cheap, with a few very expensive exceptions.

```
print("Bivariate Analysis: Feature vs. Survival")

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle('Bivariate Analysis with Survival', fontsize=16)

# Pclass vs. Survived
sns.barplot(ax=axes[0, 0], x='Pclass', y='Survived', data=df).set_title('Surviv

# Gender vs. Survived
sns.barplot(ax=axes[0, 1], x='Sex', y='Survived', data=df).set_title('Survival

# Embarked vs. Survived
sns.barplot(ax=axes[1, 0], x='Embarked', y='Survived', data=df).set_title('Surv

# Has_Cabin vs. Survived
sns.barplot(ax=axes[1, 1], x='Has_Cabin', y='Survived', data=df).set_title('Sur

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

```
Bivariate Analysis: Feature vs. Survival
```



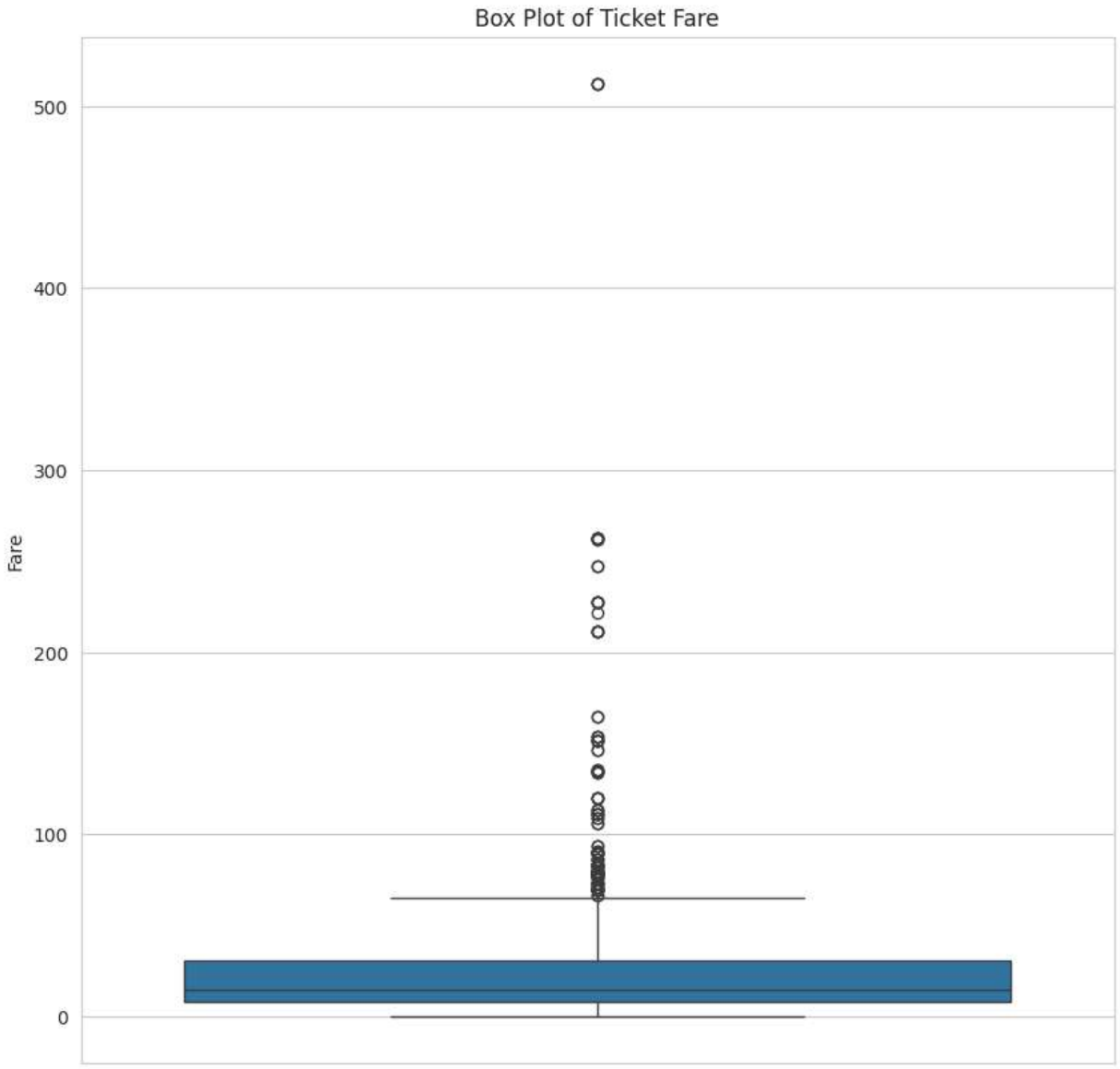
```
## Age vs Survival
g = sns.FacetGrid(df, col='Survived', height=6)
g.map(sns.histplot, 'Age', bins=25, kde=True)
plt.suptitle('Age Distribution by Survival Status', y=1.02)
plt.show()
```



Inference (Age vs. Survival):

- Infants and young children had a higher probability of survival.
- A large portion of non-survivors were young adults (20-40).
- The oldest passengers (80 years) did not survive.

```
# Deeper Dive: Outlier Analysis for 'Fare'
# The .describe() function and histogram showed that Fare has extreme outliers.
plt.figure(figsize=(10, 10))
sns.boxplot(y='Fare', data=df)
plt.title('Box Plot of Ticket Fare')
plt.ylabel('Fare')
plt.show()
```



```
# Feature Engineering - (+1 to count SELF)
# 1. Create a 'FamilySize' feature
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1

# 2. Create an 'IsAlone' Feature
df['IsAlone'] = 0
df.loc[df['FamilySize'] ==1, 'IsAlone'] = 1

df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599
				Heikkinen,					STON/O2

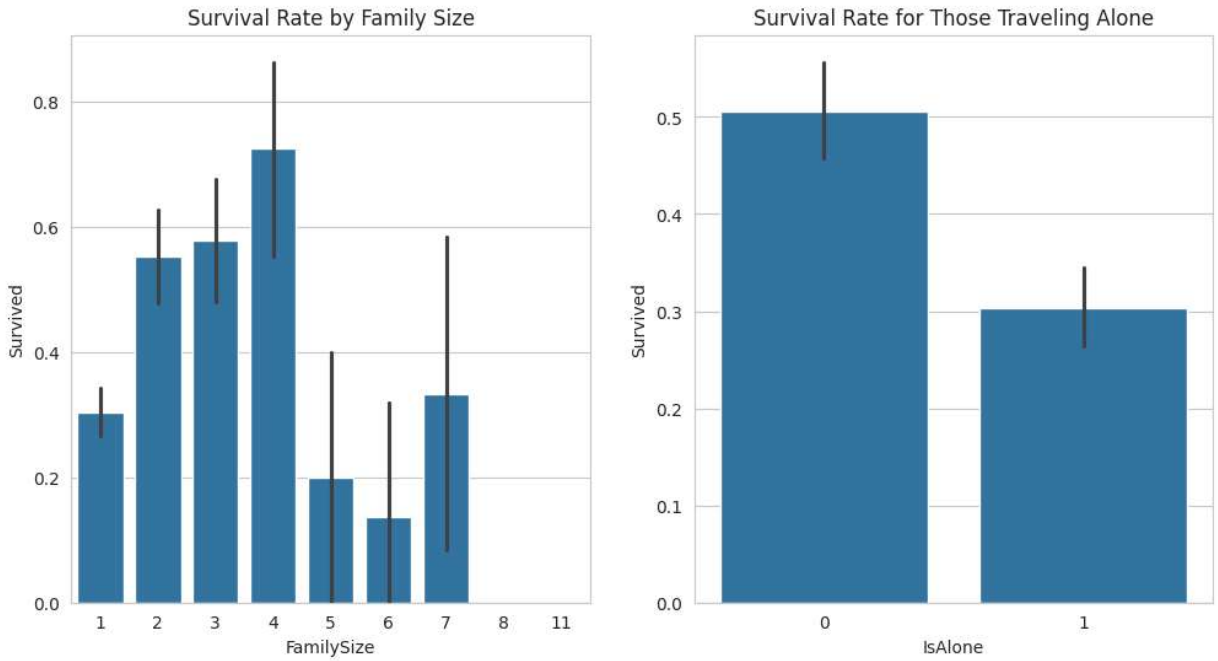
Next steps: Generate code with df | New interactive sheet

```
df.tail()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053
				Johnston,					

```
# Analyze the new family-related features against survival
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

# Survival Rate by FamilySize
sns.barplot(ax=axes[0], x='FamilySize', y='Survived', data=df).set_title('Survi

# Survival Rate by IsAlone
sns.barplot(ax=axes[1], x='IsAlone', y='Survived', data=df).set_title('Survival
```

```
plt.show()
```



Inference

- Passengers who were alone (`IsAlone=1`) had a lower survival rate (~30%) than those in small families.
- Small families of 2 to 4 members had the highest survival rates.
- Very large families (5 or more) had a very poor survival rate. This might be because it was harder for large families to stay together and evacuate.

**Survival by HONORIFICS / Title **

- Matches a space.
- Titles in the names are usually preceded by a space. ([A-Za-z]+): This is the capturing group.

- [A-Za-z]+: Matches one or more uppercase or lowercase letters. This captures the title itself (like Mr, Mrs, Miss, etc.).
- .: Matches a literal dot (.) which usually follows the title.

```
# 3. Extract 'Title' from the 'Name' column
df['Title'] = df['Name'].str.extract(r' ([A-Za-z]+)\.', expand=False)

# Let's see the different titles in the Name column
print("Extracted Titles:")
df['Title'].value_counts()
```

```
Extracted Titles:
```

Title	count
Mr	517
Miss	182
Mrs	125
Master	40
Dr	7
Rev	6
Col	2
Mlle	2
Major	2
Ms	1
Mme	1
Don	1
Lady	1
Sir	1
Capt	1
Countess	1
Jonkheer	1

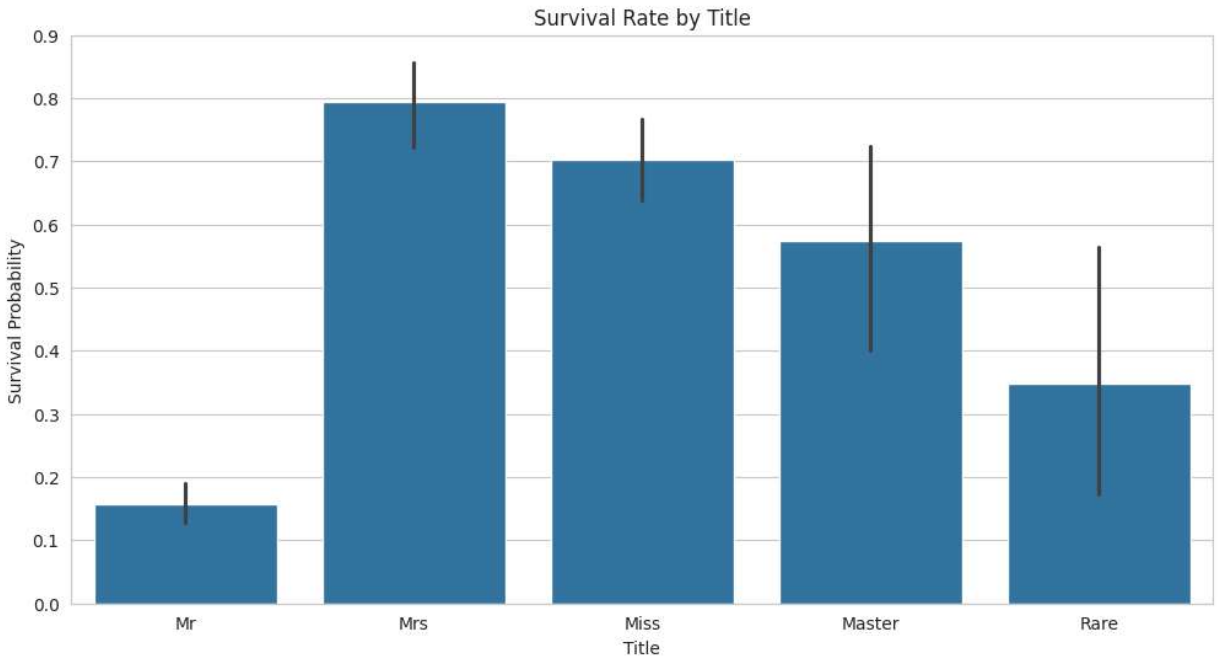
dtype: int64

```
df.head()
```

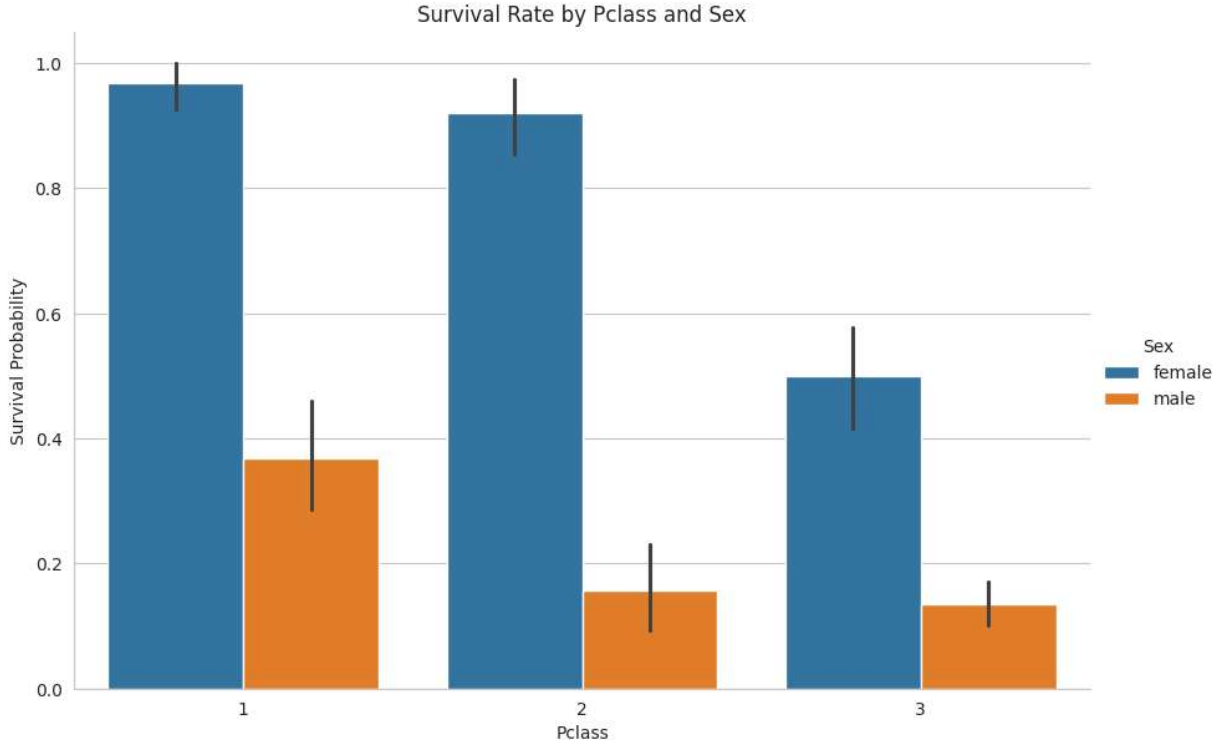
	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599
2	3	1	3	Heikkinen, Miss.	female	26.0	0	0	STON/O2.

Next steps: Generate code with df | New interactive sheet

```
# Simplify the titles by grouping rare ones into a 'Rare' category
df['Title'] = df['Title'].replace(['Lady', 'Countess','Capt', 'Col','Don', 'Dr'

df['Title'] = df['Title'].replace('Mlle', 'Miss')
df['Title'] = df['Title'].replace('Ms', 'Miss')
df['Title'] = df['Title'].replace('Mme', 'Mrs')

# Let's see the survival rate by the new, cleaned titles
plt.figure(figsize=(12, 6))
sns.barplot(x='Title', y='Survived', data=df)
plt.title('Survival Rate by Title')
plt.ylabel('Survival Probability')
plt.show()
```




```
# Multivariate Analysis
# # Survival rate by Pclass and Sex
sns.catplot(x='Pclass', y='Survived', hue='Sex', data=df, kind='bar', height=6,
plt.title('Survival Rate by Pclass and Sex')
plt.ylabel('Survival Probability')
plt.show()
```

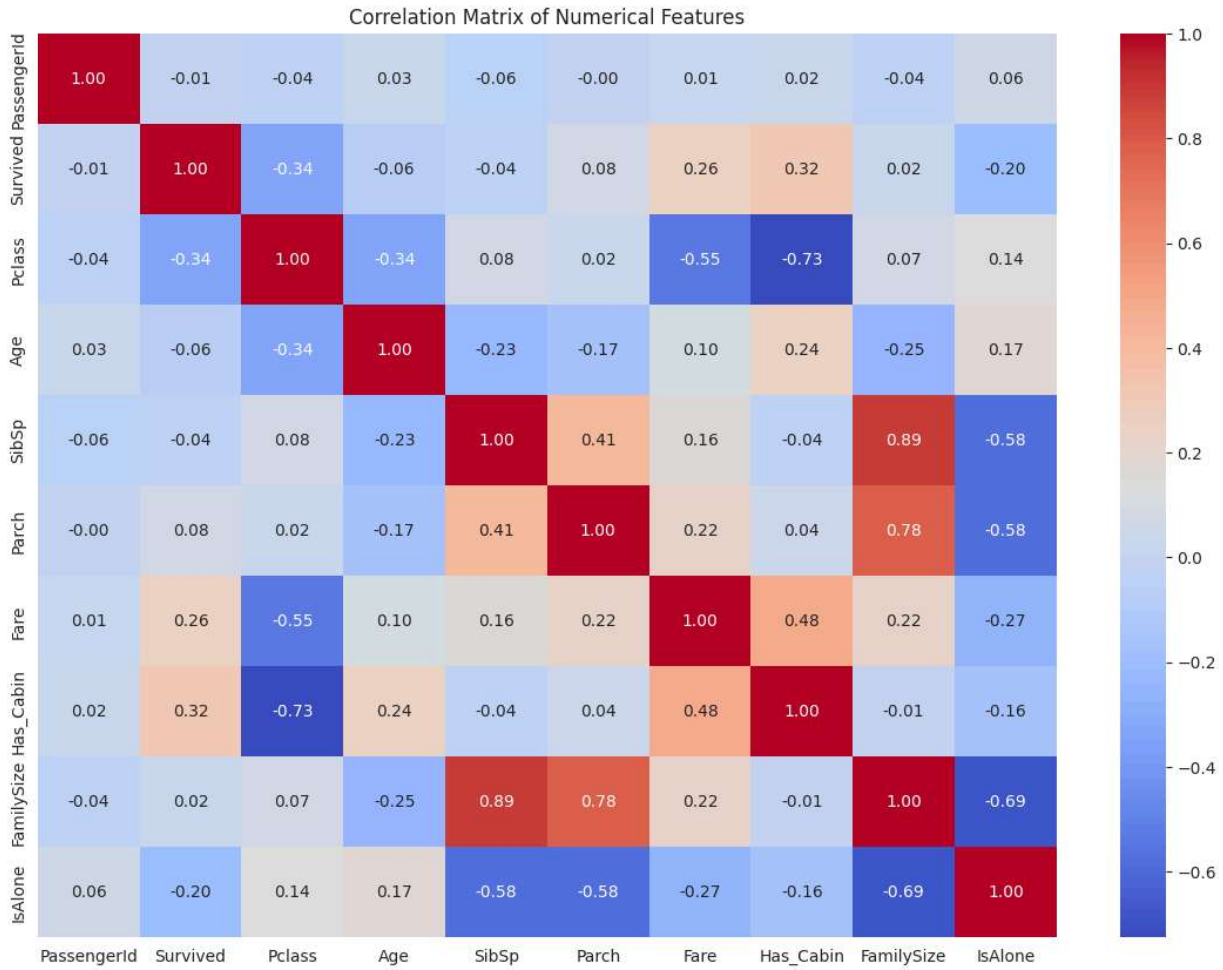


Inference of Multivariate Analysis

Females in all classes had a significantly higher survival rate than males.

```
# Violin plot to see age distribution by sex and survival status
plt.figure(figsize=(14, 8))
sns.violinplot(x='Sex', y='Age', hue='Survived', data=df, split=True, palette={
plt.title('Age Distribution by Sex and Survival')
plt.show()
```


```
# Correlation Analysis and Heatmap

# Correlation Heatmap for numerical features

plt.figure(figsize=(14, 10))
numeric_cols = df.select_dtypes(include=np.number)
correlation_matrix = numeric_cols.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix of Numerical Features')
plt.show()
```



Inference of the Heatmap:

- **`Survived`** has a notable positive correlation with `Fare` and `Has_Cabin`, and a negative correlation with `Pclass` and our new `IsAlone` feature.

- `Pclass` and `Fare` are strongly negatively correlated, which makes sense (1st class = high fare).
- Our new `FamilySize` feature is composed of `SibSp` and `Parch`, so it's highly correlated with them by definition

```
# YDATA Profiling
!pip install ydata-profiling -q
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 400.4/400.4 kB 4.5 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 296.7/296.7 kB 19.1 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.1/3.1 MB 36.2 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 679.7/679.7 kB 44.6 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 105.4/105.4 kB 7.8 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 68.0/68.0 kB 5.8 MB/s eta 0:00:00
```

```
# Generate the profiling report
from ydata_profiling import ProfileReport

profile = ProfileReport(df, title="Titanic Dataset Profiling Report")

# Display the report in the notebook
profile.to_notebook_iframe()
```

```
Summarize dataset: 100%                                  60/60 [00:05<00:00, 12.79it/s, Completed]

  0%|          | 0/15 [00:00<?, ?it/s]
 27%|██▋       | 4/15 [00:00<00:00, 35.44it/s]
100%|██████████| 15/15 [00:00<00:00, 63.35it/s]
Generate report structure: 100%                          1/1 [00:06<00:00,  6.82s/it]

Render HTML: 100%                                        1/1 [00:00<00:00,  3.13it/s]
```